



EXPERIMENT – 3

NAME: Probal Dhali
BRANCH: CSE-AIML (CS221)
SEMESTER: 5TH
SUBJECT: Full Stack - II

UID: 24BAI71013
SECTION/GROUP: 24AIT_NTPP3[B]
DATE OF PERFORMANCE: 22/07/2026
SUBJECT CODE: 24CSP-337

EXPERIMENT 3.1

1. AIM

To design and implement a secure authentication system using JWT for user login and session management.

2. OBJECTIVE

1. To understand authentication mechanisms in web applications
2. To implement token-based authentication using JWT
3. To manage user sessions in a stateless architecture
4. To handle token storage and validation securely

3. SOFTWARE REQUIREMENT

- React.js
- Node.js (optional/mock-based initially)
- JWT libraries (conceptual understanding)
- Browser (for testing)

4. THEORY

Authentication is the process of verifying the identity of a user before granting access to system resources. Traditional session-based authentication relies on server-side session storage, which can become inefficient in distributed systems.

JWT-based authentication introduces a **stateless approach**, where the server issues a signed token containing user information. This token is stored on the client and sent with each request for verification.

A JWT consists of three parts:

- Header – Defines the algorithm
- Payload – Contains user data (claims)
- Signature – Ensures integrity and authenticity

5. IMPLEMENTATION/PROCEDURE

1. Create login form UI
2. Validate user credentials (*mock or static*)
3. Generate or simulate JWT token
4. Store token (localStorage/sessionStorage)
5. Attach token to requests
6. Decode token to extract user information

6. CODE

Authentication Service (authService.js)

```
import { generateJWT, decodeJWT } from "../utils/jwtHelper";
```

```
const users = [
```

```
  { id: 1, name: "Probal Dhali", email: "24bai71013@cuchd.in", password: "24BAI71013", role: "admin" },
```

```
  { id: 2, name: "Probal Dhali", email: "24bai71013@cuchd.in", password: "24bai71013", role: "user" },
```

```
];
```



```
export function login(email, password) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const user = users.find(u => u.email === email && u.password === password);
      if (!user) return reject(new Error("Invalid email or password"));

      const token = generateJWT({
        sub: user.id,
        email: user.email,
        name: user.name,
        role: user.role,
        iat: Math.floor(Date.now() / 1000),
        exp: Math.floor(Date.now() / 1000) + 3600,
      });

      resolve({ token, user: { ...user, password: undefined } });
    }, 600);
  });
}

export const getStoredToken = () => localStorage.getItem("auth_token");

export const setStoredToken = token =>
  token
    ? localStorage.setItem("auth_token", token)
    : localStorage.removeItem("auth_token");

export function getUserFromToken(token) {
  const payload = token && decodeJWT(token);
  return payload && payload.exp > Date.now() / 1000 ? payload : null;
}

export const logout = () => localStorage.removeItem("auth_token");
```

Login Authentication (Login.js)

```
import { useState } from "react";

export default function Login({ onLogin, loading, error }) {
  const [email, setEmail] = useState("24bai71013@cuchd.in");
  const [password, setPassword] = useState("24BAI71013");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!email.trim() || !password.trim()) return;
    onLogin(email, password);
  };

  return (
    <div className="card">
```



```
<div className="card-header">
  <span className="badge">🔒 Experiment 1.3.1</span>
  <h1>JWT Authentication</h1>
  <p>Sign in to access your dashboard</p>
</div>

{error && <div className="error-msg">✖ {error}</div>}

<form onSubmit={handleSubmit}>
  <div className="form-group">
    <label>Email</label>
    <input
      type="email"
      value={email}
      placeholder="24bai71013@cuchd.in"
      disabled={loading}
      onChange={(e) => setEmail(e.target.value)}
    />
  </div>

  <div className="form-group">
    <label>Password</label>
    <input
      type="password"
      value={password}
      placeholder="24BAI71013"
      disabled={loading}
      onChange={(e) => setPassword(e.target.value)}
    />
  </div>

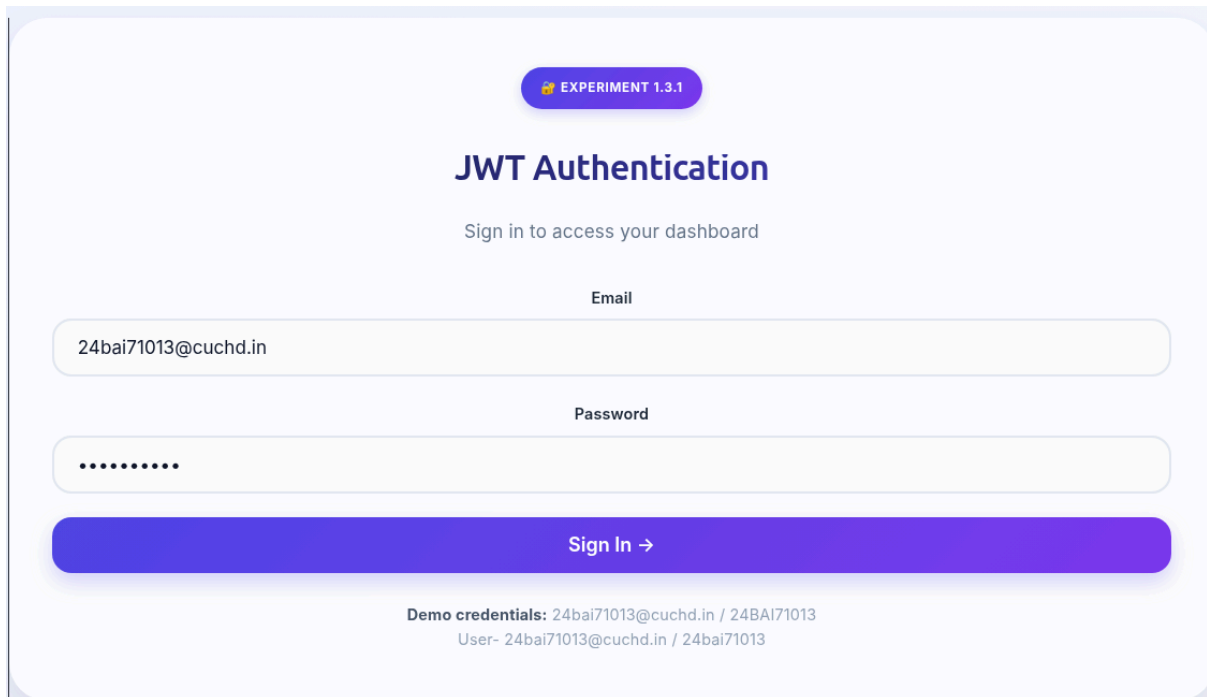
  <button className="btn" disabled={loading}>
    {loading ? "Signing in..." : "Sign In →"}
  </button>
</form>

<div className="hint">
  <strong>Admin:</strong> 24bai71013@cuchd.in / 24BAI71013
  <br />
  <strong>User:</strong> 24bai71013@cuchd.in / 24bai71013
</div>
</div>
);
}
```

7. OUTPUT

The JWT Authentication system was successfully implemented. Users were able to log in using valid credentials, after which a JWT token was generated and securely stored in the browser's localStorage. The application successfully decoded the token to retrieve user information and displayed it on the dashboard.

Invalid login attempts were rejected with an appropriate error message, and the logout functionality removed the token, terminating the user session.



The image shows a web interface for JWT Authentication. At the top, there is a purple button labeled "EXPERIMENT 1.3.1". Below it, the title "JWT Authentication" is displayed in a large, bold, dark blue font. Under the title, a subtitle "Sign in to access your dashboard" is shown in a smaller, gray font. The form consists of two input fields: "Email" and "Password". The "Email" field contains the text "24bai71013@cuchd.in". The "Password" field is masked with dots. Below the input fields is a large, blue "Sign In →" button. At the bottom of the form, there is a section for "Demo credentials" which lists the email "24bai71013@cuchd.in" and the username "24BAI71013".

Fig. 3.1 JWT Authentication system

8. LEARNING OUTCOMES

- Understand the concept of authentication in web applications.
- Implement JWT-based authentication for secure user login.
- Generate and manage JWT tokens for stateless session handling.
- Store authentication tokens securely using localStorage.
- Decode JWT tokens to retrieve user information.
- Implement login, logout, and session management functionality.
- Develop secure authentication workflows in React applications.



EXPERIMENT 3.2

1. AIM

To implement role-based access control and secure application routes based on user permissions.

2. OBJECTIVE

1. To understand authorization mechanisms in applications
2. To implement RBAC for access control
3. To protect routes based on user roles
4. To dynamically render UI elements based on permissions

3. SOFTWARE REQUIREMENT

1. React.js
2. React Router
3. State management (Context API / Redux)

4. THEORY

While authentication verifies user identity, **authorization** determines what actions a user is permitted to perform. Role-Based Access Control (RBAC) is a widely used model where users are assigned roles, and each role has defined permissions.

Common roles include:

- Admin, Editor, Viewer

RBAC ensures:

- Controlled access to sensitive features
- Simplified permission management
- Improved system security

Route protection is implemented to restrict navigation to certain pages unless the user has appropriate permissions.

5. IMPLEMENTATION/PROCEDURE

1. Define user roles (Admin, Editor, Viewer)
2. Store role in authentication state
3. Implement protected routes using React Router
4. Restrict access based on role
5. Conditionally render UI elements (buttons, pages)
6. Redirect unauthorized users

6. CODE

User Authentication (authService.js)

```
import { generateJWT, decodeJWT } from "../utils/jwtHelper";
```

```
const users = [  
  { id: 1, name: "Admin User", email: "admin.24bai71013@cuchd.in", password: "a24bai71013", role: "admin" },  
  { id: 2, name: "Editor User", email: "editor.24bai71013@cuchd.in", password: "e24bai71013", role: "editor" },  
  { id: 3, name: "Viewer User", email: "viewer.24bai71013@cuchd.in", password: "v24bai71013", role: "viewer" },  
];
```

```
export function login(email, password) {
```



```
return new Promise((resolve, reject) => {
  setTimeout(() => {
    const user = users.find(u => u.email === email && u.password === password);
    if (!user) return reject(new Error("Invalid email or password"));

    const token = generateJWT({
      sub: user.id,
      email: user.email,
      name: user.name,
      role: user.role,
      iat: Math.floor(Date.now() / 1000),
      exp: Math.floor(Date.now() / 1000) + 3600,
    });

    resolve({ token, user: { ...user, password: undefined } });
  }, 600);
});
}

export const getStoredToken = () => localStorage.getItem("auth_token");

export const setStoredToken = token =>
  token
    ? localStorage.setItem("auth_token", token)
    : localStorage.removeItem("auth_token");

export function getUserFromToken(token) {
  const payload = token && decodeJWT(token);
  return payload && payload.exp > Date.now() / 1000 ? payload : null;
}

export const logout = () => localStorage.removeItem("auth_token");
```

Login Authentication (Login.js)

```
import { useState } from "react";

export default function Login({ onLogin, loading, error }) {
  const [email, setEmail] = useState("admin.24bai71013@cuchd.in");
  const [password, setPassword] = useState("a24bai71013");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!email.trim() || !password.trim()) return;
    onLogin(email, password);
  };

  return (
    <div className="card">
      <div className="card-header">
```



```
<span className="badge">🔒 Experiment 1.3.2</span>
<h1>RBAC with JWT</h1>
<p>Sign in with different roles</p>
</div>
```

```
{error && <div className="error-msg">✖ {error}</div>}
```

```
<form onSubmit={handleSubmit}>
  <div className="form-group">
    <label>Email</label>
    <input
      type="email"
      value={email}
      placeholder="admin.24bai71013@cuchd.in"
      disabled={loading}
      onChange={(e) => setEmail(e.target.value)}
    />
  </div>

  <div className="form-group">
    <label>Password</label>
    <input
      type="password"
      value={password}
      placeholder="a24bai71013"
      disabled={loading}
      onChange={(e) => setPassword(e.target.value)}
    />
  </div>

  <button className="btn" disabled={loading}>
    {loading ? "Signing in..." : "Sign In →"}
  </button>
</form>
```

```
<div className="hint">
  <strong>Admin:</strong> admin.24bai71013@cuchd.in / a24bai71013
  <br />
  <strong>Editor:</strong> editor.24bai71013@cuchd.in / e24bai71013
  <br />
  <strong>Viewer:</strong> viewer.24bai71013@cuchd.in / v24bai71013
</div>
</div>
);
}
```

Protected Route (ProtectedRoute.js)

```
import { Navigate } from "react-router-dom";
```



```
export default function ProtectedRoute({ children, allowedRoles, user }) {  
  if (!user) return <Navigate to="/login" replace />;  
  if (allowedRoles && !allowedRoles.includes(user.role))  
    return <Navigate to="/unauthorized" replace />;  
  return children;  
}
```

Route Configuration (App.js)

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";  
import { useState } from "react";  
import { useAuth } from "../hooks/useAuth";  
import Login from "../components/Login";  
import Dashboard from "../components/Dashboard";  
import AdminPanel from "../components/AdminPanel";  
import EditorPanel from "../components/EditorPanel";  
import ViewerPanel from "../components/ViewerPanel";  
import Navbar from "../components/Navbar";  
import ProtectedRoute from "../components/ProtectedRoute";  
import Unauthorized from "../components/Unauthorized";  
import "../index.css";
```

```
export default function App() {  
  const { user, loading, login, logout, isAuthenticated } = useAuth();  
  const [loginError, setLoginError] = useState(null);  
  const [loginLoading, setLoginLoading] = useState(false);  
  
  const handleLogin = async (email, password) => {  
    setLoginError(null);  
    setLoginLoading(true);  
    const res = await login(email, password);  
    setLoginLoading(false);  
    if (!res.success) setLoginError(res.error);  
  };  
  
  return (  
    <BrowserRouter>  
      {isAuthenticated && <Navbar user={user} onLogout={logout} />}  
  
    <div className="container">  
      <Routes>  
        <Route  
          path="/login"  
          element={  
            isAuthenticated ? (  
              <Navigate to="/dashboard" replace />  
            ) : (  
              <Login  
                onLogin={handleLogin}  
                loading={loginLoading || loading}  

```




```
        error={loginError}
      />
    )
  }
/>

<Route path="/unauthorized" element={<Unauthorized />} />

<Route path="/dashboard" element={<ProtectedRoute user={user}><Dashboard user={user}
/></ProtectedRoute>} />
<Route path="/admin" element={<ProtectedRoute user={user}
allowedRoles={["admin"]} ><AdminPanel /></ProtectedRoute>} />
<Route path="/editor" element={<ProtectedRoute user={user} allowedRoles={["admin",
"editor"]} ><EditorPanel /></ProtectedRoute>} />
<Route path="/viewer" element={<ProtectedRoute user={user} allowedRoles={["admin", "editor",
"viewer"]} ><ViewerPanel /></ProtectedRoute>} />

<Route path="/" element={<Navigate to="/dashboard" replace />} />
<Route path="*" element={<Navigate to="/dashboard" replace />} />
</Routes>
</div>
</BrowserRouter>
);
}
```

Role-Based Dashboards

Admin Dashboard

- User Management
- Reports
- Settings
- Logout

Editor Dashboard

- Manage Content
- Edit Articles
- View Reports
- Logout

Viewer Dashboard

- View Content
- View Reports
- Profile
- Logout

Navigation Bar

<Navbar />

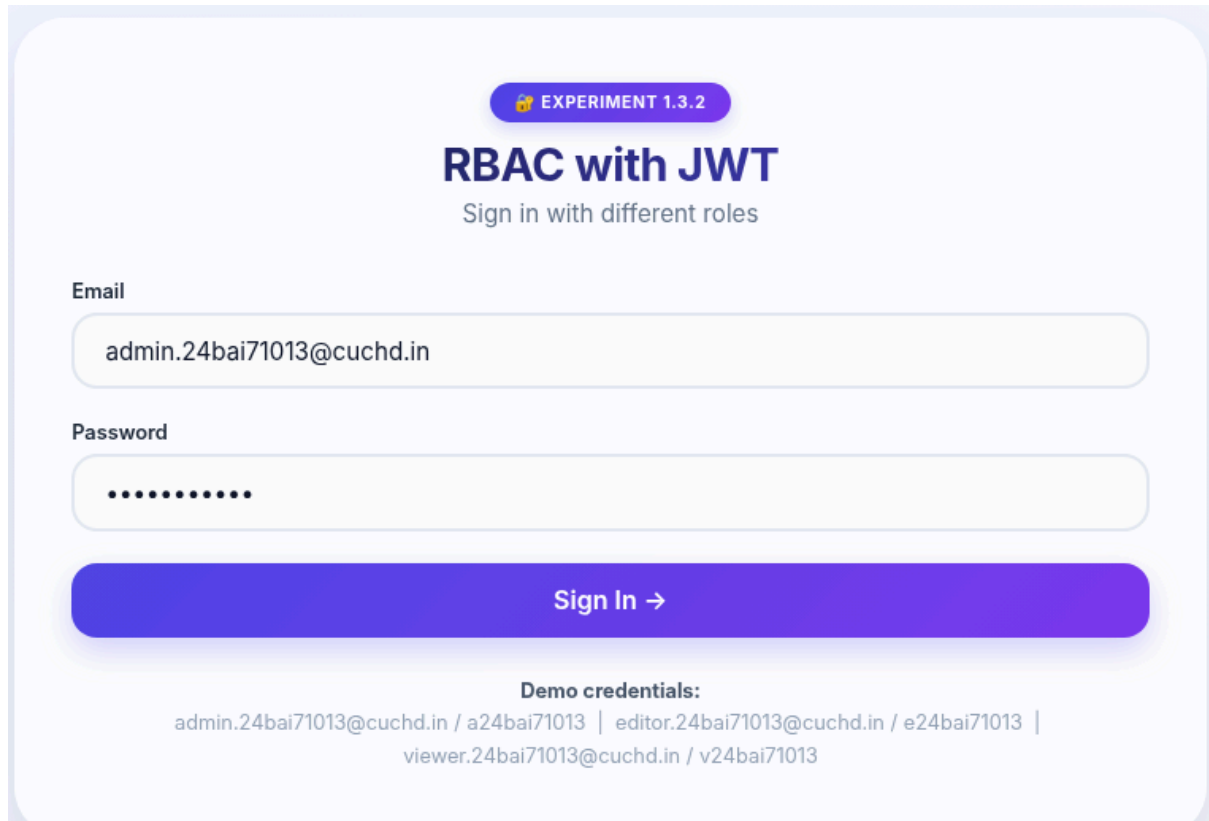
User Name

User Role

Logout Button

7. OUTPUT

The Role-Based Access Control (RBAC) system was successfully implemented. Users were authenticated using their credentials and redirected to dashboards based on their assigned roles (Admin, Editor, or Viewer). Protected routes restricted unauthorized access, and users attempting to access restricted pages were redirected to the Unauthorized (403 Access Denied) page. The application dynamically displayed different interfaces according to user permissions, demonstrating secure authorization and role-based navigation.



The screenshot shows a login interface for an RBAC system. At the top, there is a purple badge that says "EXPERIMENT 1.3.2". Below it, the title "RBAC with JWT" is displayed in a large, bold, blue font, followed by the subtitle "Sign in with different roles" in a smaller, gray font. The form consists of two input fields: "Email" and "Password". The "Email" field contains the text "admin.24bai71013@cuchd.in". The "Password" field is masked with dots. Below the input fields is a large, rounded, purple button with the text "Sign In →". At the bottom of the form, there is a section titled "Demo credentials:" followed by three lines of text: "admin.24bai71013@cuchd.in / a24bai71013", "editor.24bai71013@cuchd.in / e24bai71013", and "viewer.24bai71013@cuchd.in / v24bai71013".

Fig:3.2 Role-Based Access Control (RBAC) system

8. LEARNING OUTCOMES

- Understand the concept of authorization and Role-Based Access Control (RBAC).
- Implement protected routes using React Router.
- Restrict application access based on user roles.
- Dynamically render user interfaces according to permissions.
- Manage user roles and secure navigation in React applications.
- Implement role-based authentication and authorization for enhanced application security.
- Develop secure, scalable, and permission-based web applications.